

PilhasDuplas.java

```
1 public class PilhasDuplas implements Pilha{
2     private int capacidade;
3     private int capacidadeMinima;
4     private Object[] array;
5     private int temporariaVermelho; //temporaria topo da pilha vermelha
6     private int temporariaPreto; //temporaria topo pilha preta
7
8     //construtor
9     public PilhasDuplas(int capacidade){
10         this.capacidade = capacidade;
11         array = new Object[capacidade];
12         temporariaVermelho = -1; //pilha vermelha começa no inicio
13         temporariaPreto = capacidade; //pilha preta começa no final
14     }
15
16     //metodos para verificar se as pilhas estão vazias
17     public boolean vermelhaVazia(){
18         return temporariaVermelho == -1;
19     }
20     public boolean pretaVazia(){
21         return temporariaPreto == capacidade;
22     }
23
24     //metodos pra verificar se estão cheias
25     private boolean estaCheio(){
26         return temporariaVermelho + 1 == temporariaPreto;
27     }
28
29     //metodo pra atualizar o tamanho do array
30     public void atualizarTamanhoArray(){
31         int novaCapacidade = capacidade * 2;
32         Object b[] = new Object[novaCapacidade];
33
34         //copia pilha vermelha a partir do inicio
35         for(int f = 0; f <= temporariaVermelho; f++){
36             b[f]=array[f];
37
38
39             int qtdPretos = capacidade - temporariaPreto; //confere a quantidade de pretos
40             int novoTP = novaCapacidade - qtdPretos; //define a nova posição para a pilha preta
41             //copia a pilha preta alocando ao final do array
42             for(int i = 0; i < qtdPretos; i++){
43                 b[novoTP + i] = array[temporariaPreto + i];
44             }
```

```

45
46     this.array = b;
47     this.temporariaPreto = novoTP;
48     this.capacidade = novaCapacidade;
49 }
50
51 public void pushPilhaVermelha(Object o){
52     // confere se a pilha vermelha bate na preta
53     if(estaCheio()){
54         //atualizar tamanho array
55         atualizarTamanhoArray();
56     }
57     array[++temporariaVermelho] = o;
58 }
59
60 public void pushPilhaPreta(Object o){
61     //confere se a pilha preta bate na vermelha
62     if(estaCheio()){
63         //atualiza tamanho array
64         atualizarTamanhoArray();
65     }
66     array[--temporariaPreto] = o;
67 }
68
69
70 private void reduzirArray() {
71     int novaCapacidade = Math.max(capacidadeMinima, capacidade /
72
73     // SÃ³ reduz se a nova capacidade for menor que a atual
74     if (novaCapacidade >= capacidade) {
75         return;
76     }
77
78     Object[] novoArray = new Object[novaCapacidade];
79
80     // Copia pilha vermelha
81     for (int i = 0; i <= temporariaVermelho; i++) {
82         novoArray[i] = array[i];
83     }
84
85     // Copia pilha preta
86     int qtdPretos = capacidade - temporariaPreto;
87     int novoTopoPreto = novaCapacidade - qtdPretos;
88
89     for (int i = 0; i < qtdPretos; i++) {
90         novoArray[novoTopoPreto + i] = array[temporariaPreto + i]
91     }
92

```

```

93         this.array = novoArray;
94         this.temporariaPreto = novoTopoPreto;
95         this.capacidade = novaCapacidade;
96
97         System.out.println("Array REDUZIDO para capacidade: " + capacidade);
98     }
99
100     // verifica se precisa reduzir
101     private void verificarReducao() {
102         int totalElementos = getTotalElementos();
103
104         // Se a utiliza  o for >= 1/3 da capacidade, reduz
105         if (capacidade > capacidadeMinima && totalElementos <= capacidade) {
106             reduzirArray();
107         }
108     }
109
110     //pop da pilha vermelha
111     public Object popPilhaVermelha() {
112         if (vermelhaVazia()) {
113             throw new PilhaVaziaExcecao("A Pilha vermelha est  vazia");
114         }
115         Object elemento = array[temporariaVermelho];
116         array[temporariaVermelho] = null; // libera refer ncia
117         temporariaVermelho--;
118
119         verificarReducao(); // verifica se precisa reduzir dps de remover
120
121         return elemento;
122     }
123
124     // pop da pilha preta
125     public Object popPilhaPreta() {
126         if (pretaVazia()) {
127             throw new PilhaVaziaExcecao("A Pilha preta est  vazia");
128         }
129         Object elemento = array[temporariaPreto];
130         array[temporariaPreto] = null; // libera refer ncia
131         temporariaPreto++;
132
133         verificarReducao(); // verifica se precisa reduzir
134
135         return elemento;
136     }
137
138
139     // metodos getters para testar
140     public int getCapacidade() {

```

```
141         return capacidade;
142     }
143
144     public int getTamanhoVermelha() {
145         return temporariaVermelho + 1;
146     }
147
148     public int getTamanhoPreta() {
149         return capacidade - temporariaPreto;
150     }
151
152     // verifica a quantidade de elementos na pilha
153     private int getTotalElementos() {
154         return (temporariaVermelho + 1) + (capacidade - temporariaPreto);
155     }
156
157     // Metodos para testar e ver arrays
158     public void mostrar(){
159         System.out.println("=== Estado das Pilhas ===");
160         System.out.println("Capacidade: " + capacidade);
161         System.out.println("Total elementos: " + getTotalElementos());
162         System.out.println("Pilha Vermelha (topo â†’ base): " + toStringVermelha());
163         System.out.println("Pilha Preta (topo â†’ base): " + toStringPreta());
164         System.out.println("Array completo: " + java.util.Arrays.toString(array));
165         System.out.println("---");
166     }
167
168     // Converte a pilha vermelha para string
169     public String toStringVermelha() {
170         StringBuilder sb = new StringBuilder("[");
171         for (int i = 0; i <= temporariaVermelho; i++) {
172             if (i > 0) sb.append(", ");
173             sb.append(array[i]);
174         }
175         sb.append("]");
176         return sb.toString();
177     }
178
179     // Converte a pilha preta para string
180     public String toStringPreta() {
181         StringBuilder sb = new StringBuilder("[");
182         for (int i = capacidade - 1; i >= temporariaPreto; i--) {
183             if (i < capacidade - 1) sb.append(", ");
184             sb.append(array[i]);
185         }
186         sb.append("]");
187         return sb.toString();
188     }
```

```
189     }  
190  
191 }
```